

INTERNSHIP PROJECT REPORT

WAKEWISE

Intelligent Cognitive Alarm Platform

An AI-assisted platform combining adaptive alarms, cognitive challenges, wake-up verification and behavioral analytics.

PREPARED BY Ronak Jat

1. Introduction

A conventional alarm generally performs one simple task: it alerts the user at a selected time. Although this approach is familiar and convenient, it provides very little interaction after the alarm rings. A user can often dismiss or snooze the alarm immediately, and the system normally has no understanding of whether the user is actually awake, how consistently they wake up, or whether their behavior is improving over time.

WakeWise was developed to explore a more intelligent approach. The platform turns the wake-up event into a short interactive process in which the user may need to complete a cognitive challenge and pass wake-up verification. The system records meaningful interactions and uses them to understand performance, support adaptive difficulty, and provide personalized feedback.

The project also demonstrates a complete modern web application architecture. It connects a responsive browser-based frontend with a FastAPI backend, PostgreSQL database, authentication services, an AI challenge layer, scheduled alarm processing, analytics, and role-specific workflows.

2. Problem Statement

Traditional alarm applications are primarily notification-based. They notify the user, but they generally do not verify whether the person is sufficiently awake to continue their day. Repeated snoozing, inconsistent wake-up times, and the absence of useful behavioral feedback can make it difficult for users to understand and improve their routines.

Another limitation is the lack of personalization. A fixed alarm and identical challenge experience does not account for differences between users or changes in the same user's performance over time. WakeWise addresses this by combining alarm behavior and cognitive challenge performance with adaptive logic.

3. Project Objectives

The major objectives of WakeWise were to:

- Develop an alarm system that supports recurring, one-time, and adaptive scheduling.
- Introduce cognitive challenges as an active wake-up interaction rather than relying only on a dismiss button.
- Implement wake-up verification so that completing the required interaction becomes part of the alarm workflow.
- Track challenge attempts, accuracy, completion, snooze behavior, and wake-up outcomes.
- Use behavioral information to support adaptive challenge difficulty and personalized recommendations.
- Provide dashboards that allow users to understand their progress and allow authorized coaches or administrators to manage relevant workflows.
- Implement secure authentication using Google OAuth and JWT-based sessions.
- Deploy the application using a cloud-based frontend, backend, and PostgreSQL database.

4. Project Scope

The scope of the project covers the complete digital wake-up workflow, from account authentication and alarm creation through alarm triggering, challenge completion, verification, data recording, analytics, and recommendations.

The project includes three major access roles: User, Coach, and Admin. Users interact directly with alarms and challenges. Coaches can access permitted user information and provide notifications or guidance within assigned relationships. Administrators manage platform-level operations. Access is controlled through authentication and role-based authorization.

The project is intentionally structured so that additional intelligence, mobile integrations, wearables, and smart-home features can be introduced later without redesigning the complete application.

5. System Architecture

WakeWise follows a layered architecture in which the frontend is responsible for user interaction, the FastAPI backend coordinates application logic, the AI layer provides challenge-generation and personalization capabilities, and PostgreSQL stores persistent application data.

5.1 Presentation Layer

The presentation layer is implemented using HTML, CSS, and JavaScript. It provides the login and registration interfaces, alarm management screens, cognitive challenge interface, dashboard, analytics views, notifications, and role-specific coach and administrator interfaces.

5.2 Application Layer

The FastAPI backend acts as the central application layer. It exposes REST APIs and coordinates authentication, alarm management, scheduler operations, challenge generation and validation, notification delivery, analytics, coach-user access, and database operations.

5.3 Intelligence Layer

The AI layer supports cognitive challenge generation and personalization. Gemini is used when available for dynamic challenge generation, while a local fallback mechanism provides predefined or locally generated challenges when external AI generation is unavailable or delayed. This improves reliability and prevents the wake-up workflow from depending entirely on a third-party service.

5.4 Data Layer

PostgreSQL stores persistent application data, including user accounts, alarms, challenge attempts, notifications, snooze events, platform announcements, notification preferences, and coach-user assignments. SQLAlchemy provides the database interaction layer.

5.5 Security Layer

Google OAuth provides an external identity-based login option, while JWT tokens are used for secure session and API access. Protected routes use role and assignment checks to prevent unauthorized access to sensitive functionality.

Architecture Flow

User → Web Frontend → FastAPI REST APIs → Application Services → AI / PostgreSQL → Personalized Wake-Up & Analytics

6. Technologies and Tools

The project uses a practical full-stack technology stack selected to support rapid development, maintainability, secure authentication, database persistence, AI integration, and cloud deployment.

- Frontend: HTML5, CSS3, and JavaScript for the browser-based interface.
- Backend: Python with FastAPI for REST APIs and application services.
- Database: PostgreSQL for structured persistent storage.
- Authentication: Google OAuth and JWT-based authentication and authorization.
- AI: Gemini API with a local fallback challenge mechanism.
- Development: VS Code, Git, and GitHub.
- Deployment: Vercel for the frontend and Railway for the backend and PostgreSQL environment.
- Testing: Automated tests, endpoint verification, integration checks, and production validation.

7. Description of Work

7.1 User Authentication & Role-Based Access

Overview: Provides secure access to WakeWise through authentication and role-based permissions for User, Coach, and Admin.

Work Performed:

- Implemented Google OAuth and JWT authentication.
- Added protected API routes and role-based access.
- Connected authentication with the frontend and backend.

7.2 User Profile & Habit Management

Overview: Maintains user information and activity required for personalization.

Work Performed:

- Connected user accounts with database records.
- Linked user activity, alarm history, and challenge performance.
- Used stored data for personalization and habit analysis.

7.3 Alarm Scheduling System

Overview: Manages alarms and different scheduling patterns.

Work Performed:

- Implemented daily, weekday, weekend, one-time, and Smart Adaptive alarms.
- Developed the background scheduler.
- Connected alarm triggers with the cognitive challenge workflow.

7.4 Cognitive Challenge Engine

Overview: Provides interactive cognitive challenges during the wake-up process.

Work Performed:

- Implemented Math, Logic, Memory, Word, Pattern, Riddle, and Quiz challenges.
- Added five difficulty levels: Beginner, Easy, Medium, Hard, and Expert.
- Integrated AI-assisted generation with fallback logic.
- Implemented answer validation and attempt recording.

7.5 Adaptive Difficulty Engine

Overview: Adjusts challenge difficulty according to user performance.

Work Performed:

- Tracked accuracy, completion, and performance.

- Implemented difficulty adjustment logic.
- Connected previous performance with future challenge selection.

7.6 Wake-Up Verification Module

Overview: Uses multiple cognitive challenges to verify that the user has completed the wake-up process.

Work Performed:

- Implemented a three-step verification workflow.
- Added unique challenges for each step.
- Implemented answer validation and completion tracking.
- Connected verification results with performance records.

7.7 Behavioral Analytics Engine

Overview: Analyzes historical alarm and challenge activity to identify behavioral patterns.

Work Performed:

- Tracked wake-up consistency, snoozing, challenge accuracy, and completion.
- Aggregated historical activity.
- Connected analytics with dashboard visualizations.

7.8 Habit Scoring Engine

Overview: Provides a score representing routine and wake-up consistency.

Work Performed:

- Combined relevant behavioral indicators.
- Implemented habit-related scoring logic.
- Displayed habit progress through the dashboard.

7.9 Recommendation Engine

Overview: Provides personalized suggestions using available behavioral data.

Work Performed:

- Connected analytics and habit information with recommendation logic.
- Implemented rule-based personalization.
- Integrated recommendations into the user experience.

7.10 Dashboard & Analytics

Overview: Provides the main interface for alarms, progress, analytics, and quick actions.

Work Performed:

- Developed the HTML, CSS, and JavaScript dashboard.
- Connected dashboard components with FastAPI APIs.
- Added alarm, challenge, habit, and analytics views.
- Implemented API loading and error handling.

7.11 Notification & Reminder System

Overview: Handles application notifications and reminders.

Work Performed:

- Implemented notification of records and user preferences.
- Connected notifications with application events.

- Added duplicate prevention and notification workflow handling.

7.12 Reports & Export System

Overview: Organizes performance information for analysis and reporting.

Work Performed:

- Structured alarm, challenge, verification, and habit data for reporting.
- Integrated performance information into dashboard analytics.
- Designed the data layer for future dedicated export functionality.

7.13 Final Integration, Testing & Deployment

Overview: Integrated all modules into a complete production application.

Work Performed:

- Connected frontend, FastAPI backend, PostgreSQL, authentication, scheduler, and AI services.
- Performed functional and end-to-end testing.
- Resolved API, CORS, authentication, scheduler, and deployment issues.
- Deployed the frontend on Vercel and backend/database on Railway.

8. Development Approach

The development of WakeWise followed a modular and iterative approach. The system was developed by dividing the project into independent functional modules and integrating them progressively.

The major development steps were:

1. **Requirement Analysis** – Identified the requirements for intelligent alarms, cognitive challenges, adaptive difficulty, habit tracking, and analytics.
2. **System Design** – Designed the frontend, backend, database, authentication, scheduler, and AI components.
3. **Backend Development** – Developed REST APIs using FastAPI and connected them with PostgreSQL.
4. **Frontend Development** – Built the user interface using HTML, CSS, and JavaScript and integrated it with backend APIs.
5. **AI Integration** – Integrated AI-based challenge generation with fallback mechanisms for reliability.
6. **Feature Integration** – Connected alarms, challenges, verification, analytics, recommendations, and notifications into a unified workflow.
7. **Testing & Debugging** – Tested individual modules and complete user workflows, resolving API, authentication, CORS, scheduling, and deployment issues.
8. **Deployment** – Deployed the frontend and backend using Vercel and Railway and verified the application in a production environment.

9. User Workflow

A typical WakeWise session begins when a user creates an alarm and selects the required schedule. The alarm is stored in PostgreSQL and evaluated by the backend scheduler.

- The scheduler detects that an active alarm is due.
- A wake-up verification session is prepared for the alarm occurrence.
- An appropriate cognitive challenge is selected or generated.
- The user receives the wake-up interaction through the frontend.
- The user submits an answer and the backend validates it.
- The verification workflow progresses according to the configured steps.
- The result, challenge attempt, and relevant alarm behavior are stored.
- Analytics and habit-related indicators are updated.
- Future challenge selection and recommendations can use the accumulated behavioral information.

10. Security and Reliability

Security was treated as a core application requirement rather than an additional feature. Authentication and authorization are handled separately: authentication establishes the identity of the user, while authorization determines which resources that user is allowed to access.

- Google OAuth provides a secure external login mechanism.
- JWT tokens are used for authenticated API access.
- Role-based checks protect User, Coach, and Admin functionality.
- Coach-user assignments are validated before protected coach operations are allowed.
- Sensitive credentials are kept in environment configuration rather than application source code.
- CORS configuration restricts browser access to approved production and development origins.
- AI generation includes a fallback path to reduce dependence on an external service.
- Notification and email delivery are designed so that external delivery failures do not automatically invalidate core in-app functionality.

11. Challenges Encountered and Solutions

11.1 Adaptive Challenge Selection

One challenge was deciding how to make the system adaptive without making it unstable. A single incorrect answer should not necessarily cause a large difficulty reduction. The implementation therefore considers accumulated performance and challenge history when selecting future difficulty.

11.2 AI Service Reliability

External AI generation can experience timeouts or model availability issues. Because a wake-up system needs predictable behavior, a local fallback mechanism was included so the user can still receive a challenge.

11.3 Scheduler and Time Handling

Time-sensitive applications require careful handling of server time, user time, recurring schedules, and trigger state. The scheduler was designed to evaluate alarms against the appropriate user time and to maintain trigger state, so the same alarm occurrence is not unintentionally processed repeatedly.

11.4 Authentication and CORS

During integration, authentication and browser cross-origin communication required careful coordination between the deployed frontend and backend. Production origins were explicitly configured, and API preflight behavior was tested to ensure the browser could communicate with protected endpoints.

11.5 Notification Delivery

Notification delivery involved both in-app records and optional external email delivery. Network-level SMTP failures highlighted the importance of separating these responsibilities. The system was structured so that an external email failure should not prevent an in-app notification from being created.

11.6 Database and Deployment Consistency

Moving the application from a local PostgreSQL environment to a cloud PostgreSQL service required preserving the schema and application data while updating deployment configuration. Database restoration and verification were performed before production use.

12. Application Screenshots

The final version of the report should include screenshots of the major interfaces. Use consistent image sizes, captions, and spacing. Recommended screenshots are shown below as placeholders.

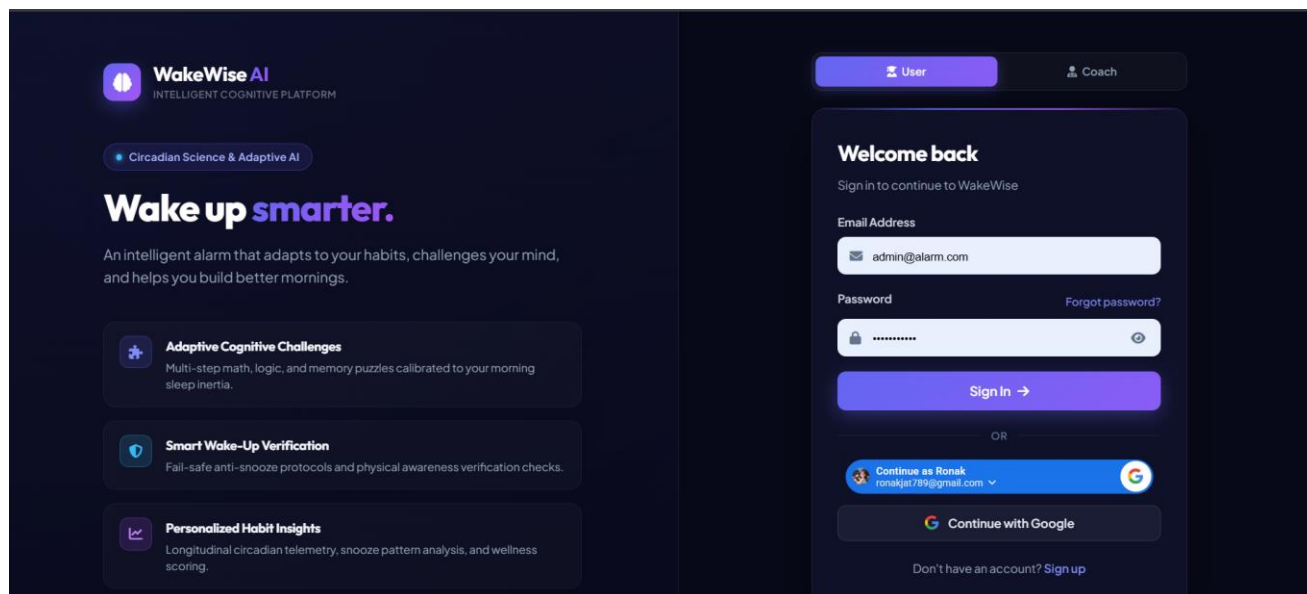


Figure 1. WakeWise authentication interface with email/password and Google OAuth.

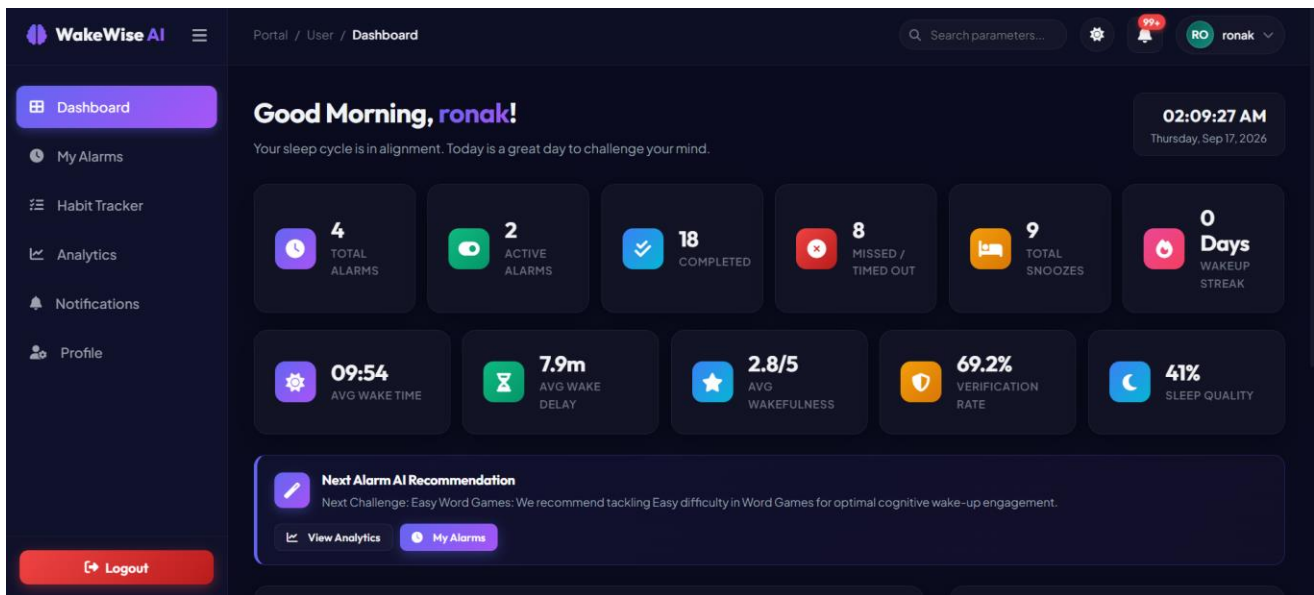


Figure 2. User dashboard showing alarms, progress, habit indicators, and recommendations.

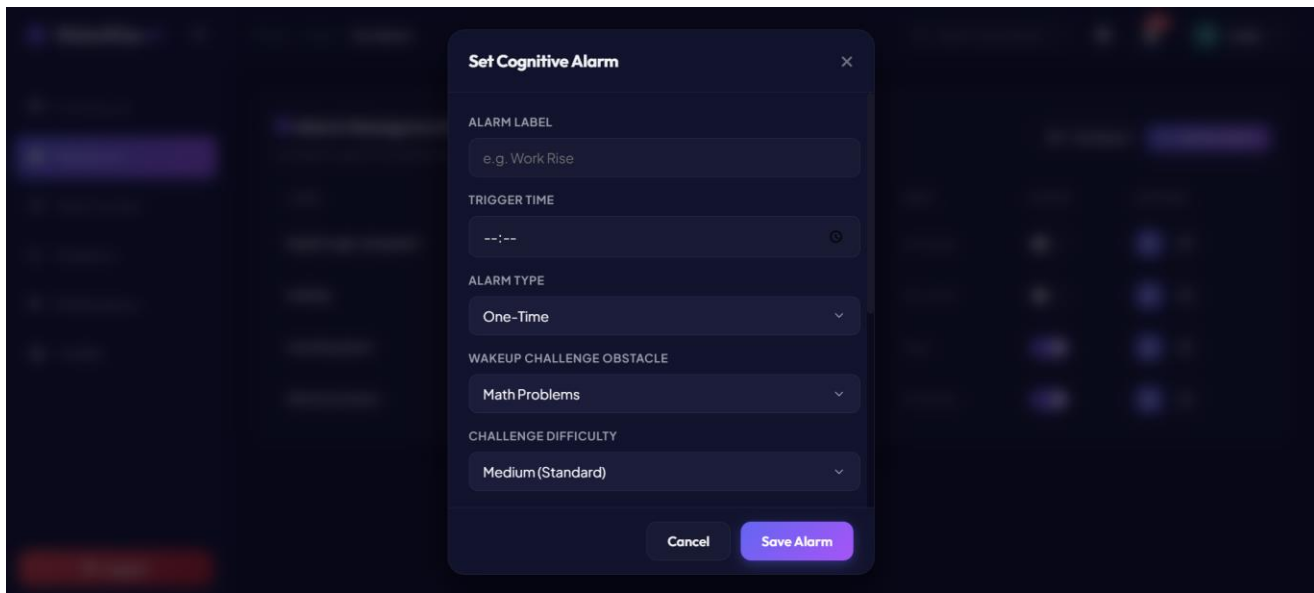


Figure 3. Interface for creating a cognitive alarm.

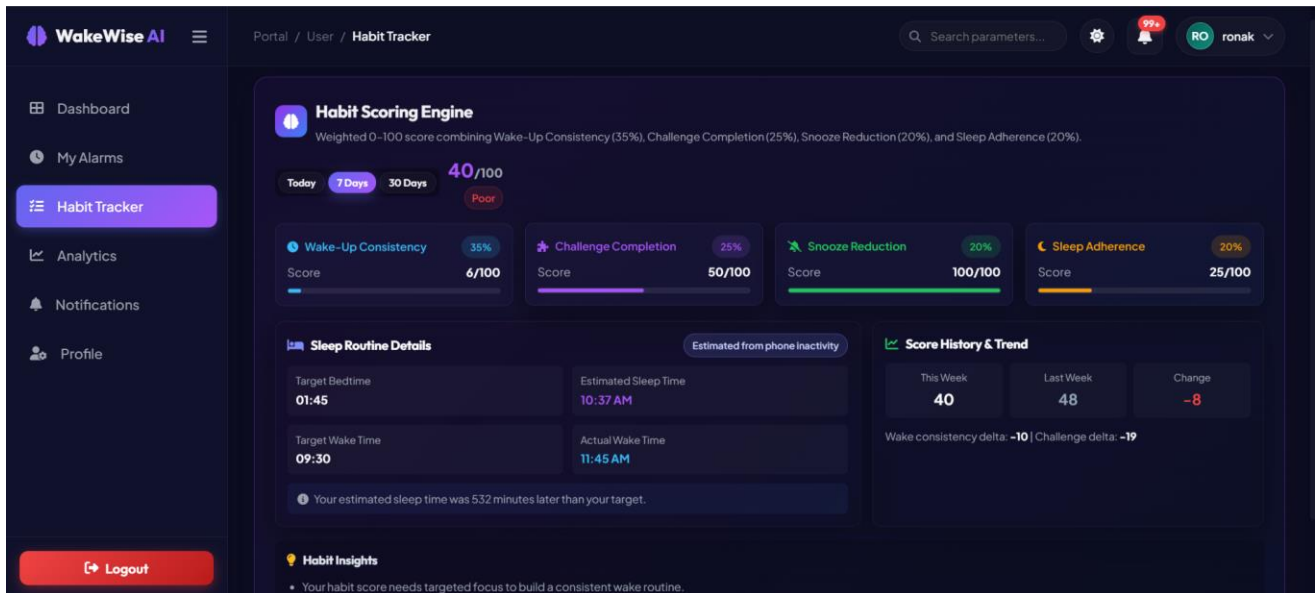


Figure 4. Behavioral analytics, challenge performance, and habit trends.

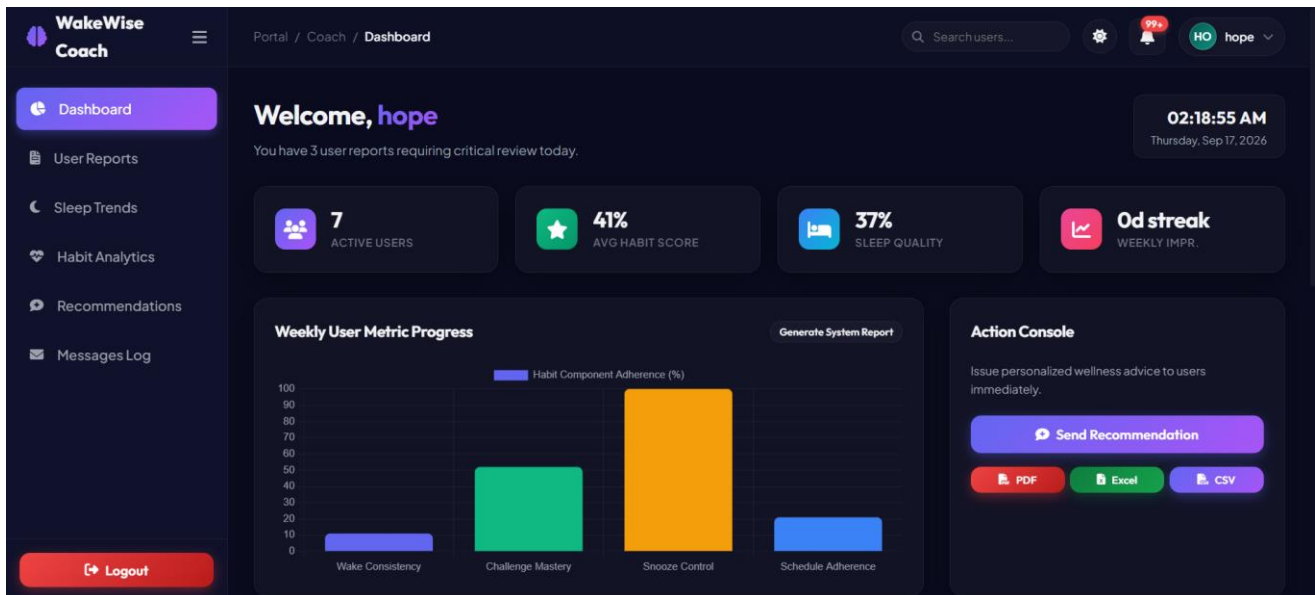


Figure 5. Coach dashboard interface.



Figure 6. Admin dashboard interface.

15. Learnings and Skills Acquired

The project provided practical experience across software engineering, AI integration, databases, deployment, security, testing, and technical communication.

- Full-stack application development using a browser frontend and Python backend.
- REST API design and integration using FastAPI.
- Relational database design and PostgreSQL administration.
- Google OAuth and JWT authentication.
- Role-based authorization and protected API design.
- AI integration with fallback and reliability considerations.
- Scheduled background processing and time-sensitive application logic.
- Cloud deployment using Vercel and Railway.
- Debugging production issues involving CORS, authentication, databases, scheduler behavior, and external services.

16. Future Enhancements

WakeWise provides a foundation for several future improvements. These enhancements would extend the platform beyond a web prototype and provide richer signals for personalization.

- Native Android/iOS application with more reliable background alarm execution.
- Integration with smartwatches and wearable devices for sleep and activity information.
- Voice-based wake-up interactions and conversational cognitive challenges.
- Predictive models that estimate the likelihood of oversleeping based on historical behavior.
- Smart-home integration with lights, speakers, and environmental controls.
- Multi-language challenge generation and improved accessibility.
- Offline-first challenge support for situations with poor connectivity.
- Production-grade distributed scheduling if the application is scaled to multiple backend instances.

18. Conclusion

WakeWise demonstrates a practical approach to building an intelligent alarm system around interaction, verification, behavioral measurement, and personalization. The project brings together alarm scheduling, cognitive challenges, adaptive difficulty, wake-up verification, analytics, habit scoring, recommendations, notifications, secure authentication, and role-based workflows within a single full-stack platform.

Beyond the individual features, the project provided valuable experience in designing and integrating a complete software system. It required solving problems across frontend development, backend APIs, databases, authentication, background processing, AI reliability, cloud deployment, testing, and production debugging.

The resulting platform provides a strong foundation for future work involving mobile alarms, wearable data, predictive personalization, smart-home integration, and controlled evaluation of user outcomes.

19. Acknowledgement

I would like to express my sincere gratitude to my mentor, coordinators, team members, and everyone who supported the development of WakeWise. Their guidance, feedback, and technical discussions helped me understand how concepts learned during my academic studies can be applied to a practical software project.

This internship provided an opportunity to experience the complete development lifecycle, from identifying a problem and planning the architecture to implementing features, debugging integration issues, deploying the application, testing production behavior, and documenting the final system.

20. References and Supporting Material

Repository: <https://github.com/ronak-jat/wakewise>

Frontend Deployment: [Vercel – WakeWise](#)

API Documentation: FastAPI / Swagger interface